

Conceituando o DB para o projeto

Para o projeto *Horta Comunitária*, o MongoDB já está integrado como banco principal para telemetria e colheitas, mas o InfluxDB seria uma alternativa mais eficiente para lidar com os dados de sensores em tempo real. A escolha depende se você quer otimizar ingestão e análise temporal (InfluxDB) ou manter flexibilidade para dados contextuais e relacionais (MongoDB).

Contexto do Projeto

O repositório **Horta Comunitária** é um MVP que integra:

- **IoT (ESP32 + sensores):** coleta de umidade do solo, temperatura e controle de irrigação via MQTT.
- **Backend (Express API):** expõe endpoints para status, histórico de telemetria, irrigação e colheitas.
- **Banco de dados:** atualmente usa **MongoDB** para persistir telemetria e colheitas quando disponível.
- **Frontend (React/Expo):** painel web e aplicativo mobile para voluntários e moradores.

Comparação InfluxDB vs MongoDB para este projeto

Critério	MongoDB (atual)	InfluxDB (alternativa)
Integração existente	Já configurado no backend via MONGO_URI	Exigiria adaptação da API para usar Flux/SQL-like
Modelo de dados	Documentos JSON flexíveis (telemetria + colheitas + reservas)	Séries temporais otimizadas para métricas de sensores
Desempenho em telemetria	Bom para volumes moderados, mas overhead maior	Excelente para ingestão massiva e consultas temporais

Consultas	Flexíveis, mas menos eficientes para análises temporais	Agregações temporais rápidas (médias, máximos, tendências)
Armazenamento	Mais pesado para séries temporais longas	Compressão nativa → menor custo
Casos de uso ideais	Dados mistos: telemetria + usuários + colheitas	Telemetria pura: sensores de solo, temperatura, irrigação

Recomendação para a Horta Comunitária

- **Manter MongoDB** para dados **contextuais** (colheitas, reservas, perfis de voluntários, autenticação).
- **Adicionar InfluxDB** para **telemetria contínua** dos sensores (umidade, temperatura, acionamento da irrigação). Isso traria:
 - Melhor desempenho em consultas históricas (ex.: gráficos de umidade ao longo da semana).
 - Redução de custo de armazenamento para grandes volumes de dados.
 - Facilidade em implementar alertas e relatórios de eficiência hídrica.

Pontos de atenção

- **Complexidade extra:** usar dois bancos exige integração e sincronização no backend.
- **Escopo do MVP:** como o projeto está em fase inicial, manter apenas MongoDB simplifica a arquitetura.
- **Evolução futura:** se o número de sensores e leituras crescer muito, migrar a telemetria para InfluxDB será vantajoso.

 Em resumo: **MongoDB é suficiente para o MVP atual**, mas se a horta comunitária escalar para dezenas de sensores e leituras contínuas, **InfluxDB deve ser incorporado para telemetria** enquanto o MongoDB continua cuidando dos dados relacionais e administrativos.

Arquitetura Híbrida IoT para a Horta Comunitária

1. Camada de Dispositivos (IoT)

- **ESP32 + Sensores:** coletam dados de umidade, temperatura e acionam irrigação.
- **MQTT Broker:** recebe mensagens dos dispositivos e distribui para o backend.

2. Camada de Backend (API/Serviços)

- **Express API:**
 - Recebe dados via MQTT.
 - Encaminha telemetria para **InfluxDB**.
 - Persiste dados contextuais (colheitas, voluntários, reservas) em **MongoDB**.
 - Expõe endpoints REST para frontend e mobile.

3. Camada de Bancos de Dados

- **InfluxDB (telemetria):**
 - Armazena leituras contínuas dos sensores.
 - Permite consultas temporais rápidas (gráficos de tendência, alertas).
 - Ideal para dashboards de monitoramento em tempo real.
- **MongoDB (dados contextuais):**
 - Armazena informações de usuários, colheitas, reservas e histórico de irrigação.
 - Flexível para dados não estruturados e relacionais.

4. Camada de Visualização

- **Frontend Web (React):**
 - Painel para voluntários monitorarem a horta.
 - Gráficos de telemetria (via InfluxDB).
 - Gestão de colheitas e reservas (via MongoDB).
- **Mobile App (Expo/React Native):**
 - Interface simplificada para moradores e voluntários.
 - Alertas de irrigação e disponibilidade de colheitas.



Benefícios da Arquitetura Híbrida

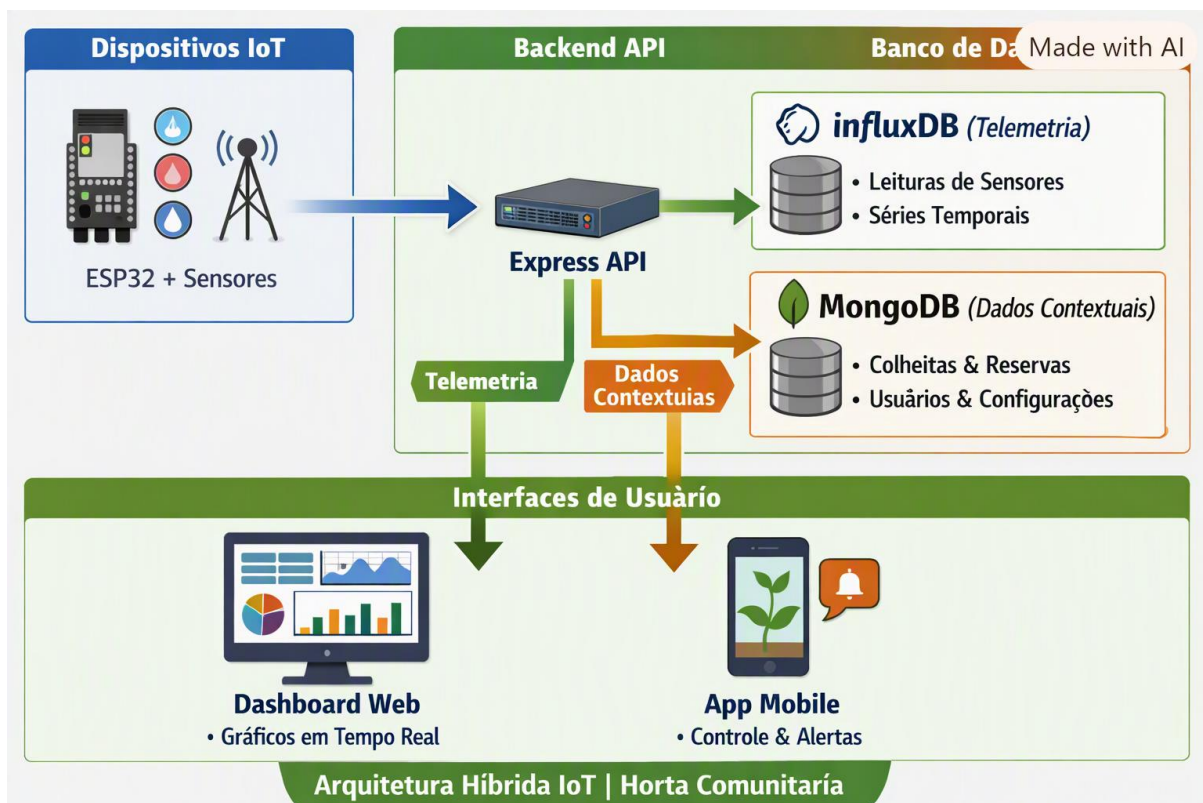
- **Desempenho:** InfluxDB otimiza ingestão e análise de dados de sensores.
- **Flexibilidade:** MongoDB mantém dados administrativos e contextuais.
- **Escalabilidade:** separação clara entre telemetria massiva e dados relacionais.
- **Manutenção:** cada banco cumpre seu papel sem sobrecarga desnecessária.



Exemplo de Fluxo

1. Sensor envia leitura de umidade → MQTT → API → InfluxDB.
2. Voluntário registra colheita → API → MongoDB.
3. Dashboard mostra gráfico de umidade (InfluxDB) e lista de colheitas (MongoDB).
4. Sistema gera alerta de irrigação automática baseado em regra temporal (InfluxDB).

👉 Assim, o projeto ganha **robustez e eficiência**: MongoDB continua sendo o coração administrativo, enquanto InfluxDB se torna o motor de telemetria em tempo real.



Aqui está o diagrama da **arquitetura híbrida IoT para a Horta Comunitária** que você pediu — ele mostra como MongoDB e InfluxDB podem coexistir no mesmo projeto, cada um cumprindo um papel específico.

Como interpretar o diagrama

- **Dispositivos IoT (ESP32 + Sensores)** → enviam dados de umidade e temperatura via **MQTT Broker**.
- **Backend (Express API)** → recebe os dados e distribui:
 - **InfluxDB** para telemetria contínua (leituras de sensores).
 - **MongoDB** para dados contextuais (colheitas, reservas, usuários).
- **Interfaces de Usuário** →
 - **Dashboard Web** mostra gráficos em tempo real (InfluxDB).
 - **App Mobile** exibe alertas e controle de irrigação (MongoDB + InfluxDB).

Benefício da abordagem híbrida

- **InfluxDB** garante eficiência e desempenho para séries temporais (telemetria).
- **MongoDB** mantém flexibilidade para dados administrativos e relacionais.
- Juntos, oferecem uma solução escalável e robusta para a horta comunitária.

1. Instalação das dependências

No backend Node.js (Express), adicione os pacotes necessários:

```
bash
npm install mongoose influx mqtt
```

- **mongoose** → conexão e modelagem com MongoDB
- **influx** → cliente para InfluxDB
- **mqtt** → comunicação com os sensores via broker

2. Conexão com os bancos

◇ MongoDB (dados contextuais)

Js

```
import mongoose from "mongoose";

mongoose.connect(process.env.MONGO_URI, {
  useNewUrlParser: true,
  useUnifiedTopology: true,
});

const Colheita = mongoose.model("Colheita", {
  nome: String,
  quantidade: Number,
  data: Date,
});
```

◇ InfluxDB (telemetria)

Js

```
import { InfluxDB, Point } from "@influxdata/influxdb-client";

const influx = new InfluxDB({
  url: process.env.INFLUX_URL,
  token: process.env.INFLUX_TOKEN,
});

const writeApi = influx.getWriteApi(process.env.INFLUX_ORG,
process.env.INFLUX_BUCKET);
```

3. Recebendo dados via MQTT

Js

```
import mqtt from "mqtt";

const client = mqtt.connect(process.env.MQTT_BROKER);
```

```

client.on("message", (topic, message) => {
  const data = JSON.parse(message.toString());
  const point = new Point("sensor_data")
    .tag("sensor", data.id)
    .floatField("umidade", data.umidade)
    .floatField("temperatura", data.temperatura)
    .timestamp(new Date());

  writeApi.writePoint(point);
});

```

4. Endpoints Express

Js

```

import express from "express";
const app = express();

app.get("/telemetria", async (req, res) => {
  const queryApi = influx.getQueryApi(process.env.INFLUX_ORG);
  const fluxQuery = `from(bucket:"${process.env.INFLUX_BUCKET}")
    |> range(start: -1h)
    |> filter(fn: (r) => r._measurement == "sensor_data")`;
  const rows = [];
  await queryApi.collectRows(fluxQuery, rows);
  res.json(rows);
});

app.get("/colheitas", async (req, res) => {
  const colheitas = await Colheita.find();
  res.json(colheitas);
});

```

5. Benefício da integração

- **InfluxDB** → armazena e consulta dados de sensores em tempo real.
- **MongoDB** → mantém informações administrativas e contextuais.

- **Express API** → atua como ponte entre os dois mundos, simplificando o acesso para o frontend e o app mobile.

Resultado

Essa estrutura permite:

- Dashboards com gráficos de umidade e temperatura (InfluxDB).
- Gestão de colheitas e voluntários (MongoDB).
- Escalabilidade e eficiência para futuras expansões da horta.

Aqui está o **fluxo de dados completo da Horta Comunitária**, ilustrando como as mensagens MQTT percorrem o sistema até aparecerem no dashboard web e no app mobile 🌱

Fluxo de Dados IoT

1. **ESP32 + Sensores** → Captura umidade, temperatura e envia telemetria via MQTT (horta/telemetry).
2. **Broker Mosquitto** → Recebe as mensagens e as repassa para o **Backend Express API**.
3. **Backend (Express + Node.js)** → Processa os dados:
 - a. Grava telemetria no **InfluxDB** (séries temporais).
 - b. Atualiza status e colheitas no **MongoDB** (dados contextuais).
 - c. Publica comandos MQTT (horta/irrigation) para ligar/desligar o relé.
4. **Frontend Web (React + Vite)** → Consulta /status e /telemetry/history para exibir gráficos em tempo real.
5. **App Mobile (Expo)** → Mostra o mesmo status e permite acionar irrigação ou reservar colheitas.

⚙️ Resumo do Ciclo

Code

ESP32 → MQTT Broker → Backend → InfluxDB/MongoDB → Web/Mobile

Cada leitura de sensor percorre esse caminho, e cada comando de irrigação faz o caminho inverso — do painel até o relé físico.

📝 Benefícios

- **Tempo real:** dados fluem continuamente sem necessidade de polling pesado.
- **Escalabilidade:** InfluxDB absorve grandes volumes de telemetria.
- **Consistência:** MongoDB mantém o estado e histórico administrativo.
- **Interatividade:** Web e mobile refletem o mesmo estado da horta.

